EXPERIMENT – 3
CLI, GUI AND VUI

AIM:
To develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) in Python.

TOOLS REQUIRED:
• Python 3.x
• VS Code
• Windows Operating System.

THEORY:
A Command Line Interface (CLI) allows users to interact with a program using text-based commands. In this experiment, file handling is used to store tasks permanently in a text file. Functions and loops are used to manage user interaction efficiently.

1. CLI IMPLEMENTATION (COMMAND LINE INTERFACE) :

CODE:

```
# ==============================
# CLI MENU DRIVEN APPLICATION
# Student Task Manager (CLI Demo)
# ==============================

tasks = []

def show_menu():
   print("\n===== COMMAND LINE INTERFACE MENU =====")
   print("1. Add Task")
   print("2. View Tasks")
   print("3. Search Task")
   print("4. Delete Task")
   print("5. Exit")

def add_task():
   task = input("Enter new task: ")
```

```python
    tasks.append(task)
    print("Task added successfully.")

def view_tasks():
    if not tasks:
        print("No tasks available.")
    else:
        print("\nYour Tasks:")
        for i, task in enumerate(tasks, start=1):
            print(f"{i}. {task}")

def search_task():
    keyword = input("Enter keyword to search: ")
    found = False
    for task in tasks:
        if keyword.lower() in task.lower():
            print("Found:", task)
            found = True
    if not found:
        print("No matching task found.")

def delete_task():
    view_tasks()
    if tasks:
        try:
            num = int(input("Enter task number to delete: "))
            removed = tasks.pop(num - 1)
            print(f"Deleted task: {removed}")
        except:
            print("Invalid selection.")

# Main CLI Loop
while True:
    show_menu()
    choice = input("Enter your choice (1-5): ")

    if choice == "1":
        add_task()
    elif choice == "2":
        view_tasks()
    elif choice == "3":
        search_task()
```

```
elif choice == "4":
    delete_task()
elif choice == "5":
    print("Exiting CLI application...")
    break
else:
    print("Invalid choice. Try again.")
```

HOW IT WORKS :

1. The program is a Command Line Interface (CLI) based Task Manager application that allows users to manage tasks through text-based commands.
2. Tasks are temporarily stored in a list during program execution.
3. The **add_task()** function enables users to add new tasks, which are appended to the task list.
4. The **view_tasks()** function displays all available tasks with serial numbering for easy identification.
5. The **search_task()** function allows users to find tasks by entering a keyword, improving accessibility and navigation.
6. The **delete_task()** function removes a selected task from the list based on the user's input.
7. Input validation and exception handling are implemented to manage invalid selections and prevent program errors.
8. A menu-driven loop continuously displays options and allows user interaction until the Exit option is selected, demonstrating the workflow of a typical CLI application.

OUTPUT:

```
(base) shrutika@Shrutikas-MacBook-Air uid % /usr/local/bin/python3 /Users/shrutika/uid/cli.py

===== COMMAND LINE INTERFACE MENU =====
1. Add Task
2. View Tasks
3. Search Task
4. Delete Task
5. Exit
Enter your choice (1-5): 1
Enter new task: studying
Task added successfully.

===== COMMAND LINE INTERFACE MENU =====
1. Add Task
2. View Tasks
3. Search Task
4. Delete Task
5. Exit
Enter your choice (1-5): 2

Your Tasks:
1. studying

===== COMMAND LINE INTERFACE MENU =====
1. Add Task
2. View Tasks
3. Search Task
4. Delete Task
5. Exit
Enter your choice (1-5): 3
Enter keyword to search: study
Found: studying

===== COMMAND LINE INTERFACE MENU =====
1. Add Task
2. View Tasks
3. Search Task
4. Delete Task
5. Exit
Enter your choice (1-5): 4

Your Tasks:
1. studying
Enter task number to delete: 1
Deleted task: studying

===== COMMAND LINE INTERFACE MENU =====
1. Add Task
2. View Tasks
3. Search Task
4. Delete Task
5. Exit
Enter your choice (1-5): 2
No tasks available.
```

2. GUI IMPLEMENTATION (GRAPHICAL USER INTERFACE) :

THEORY:

A Graphical User Interface (GUI) allows users to interact with a program using graphical elements such as buttons, labels, and text fields instead of typing commands.

CODE:

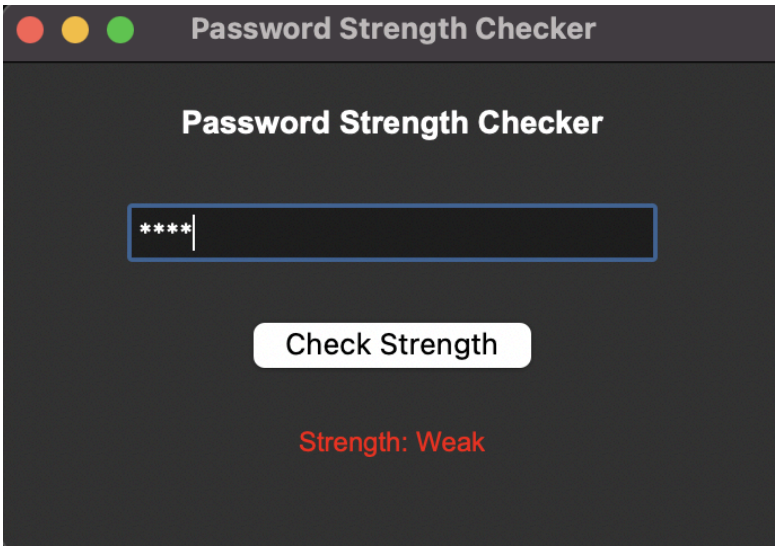
```
import tkinter as tk

import re



def check_password():

    password = entry.get()



    strength = "Weak"

    color = "red"



    if (len(password) >= 8 and

        re.search("[A-Z]", password) and

        re.search("[a-z]", password) and

        re.search("[0-9]", password) and

        re.search("[@#$%^&*!]", password)):

        strength = "Strong"

        color = "green"



    elif len(password) >= 6:

        strength = "Medium"

        color = "orange"



    result.config(text=f"Strength: {strength}", fg=color)
```

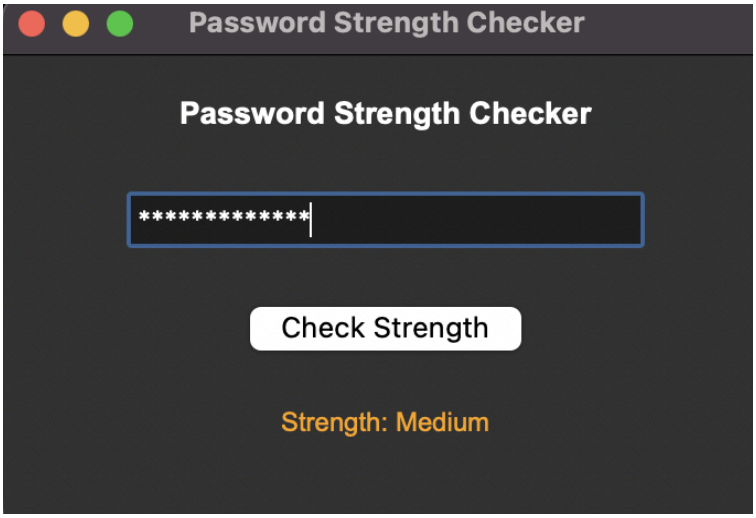
```python
# Window setup
window = tk.Tk()
window.title("Password Strength Checker")
window.geometry("350x220")

title = tk.Label(window, text="Password Strength Checker",
                font=("Arial", 14, "bold"))
title.pack(pady=15)

entry = tk.Entry(window, show="*", width=25)
entry.pack(pady=10)

check_btn = tk.Button(window, text="Check Strength",
                     command=check_password)
check_btn.pack(pady=10)

result = tk.Label(window, text="", font=("Arial", 12))
result.pack(pady=10)

window.mainloop()
```

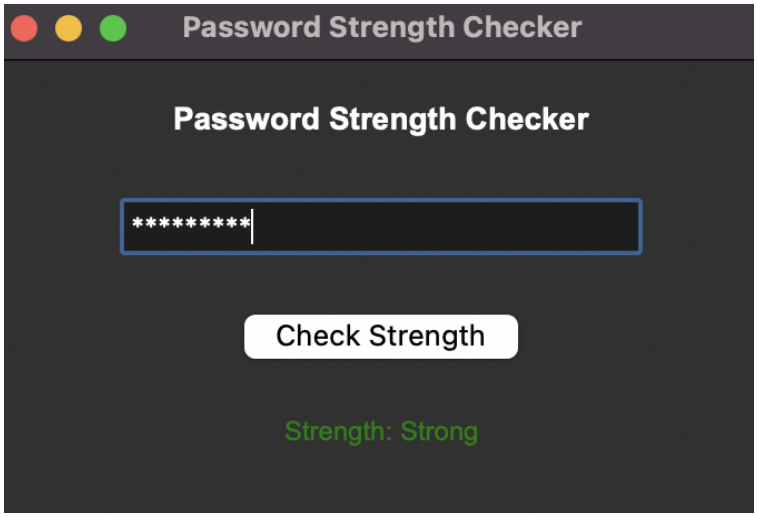
HOW IT WORKS:

1. The Tkinter library is imported to design the graphical user interface of the application.
2. A main window is created with the title **"Password Strength Checker"** and a fixed size to maintain a consistent layout.
3. An input field is provided for users to enter a password, with characters masked for security.
4. A **"Check Strength"** button is added to evaluate the entered password.
5. When the button is clicked, the check_password() function is executed.
6. The function analyzes the password using conditions such as length, uppercase letters, lowercase letters, numbers, and special characters.
7. Based on the evaluation, the password strength is classified as **Weak**, **Medium**, or **Strong**.
8. The result is displayed on the interface using a label with different colors to visually represent the strength level.
9. Input validation ensures proper handling of user entries and provides an interactive and user-friendly GUI experience.

OUTPUT:







3. VUI IMPLEMENTATION (VOICE USER INTERFACE) :

THEORY:

A Voice User Interface (VUI) allows users to interact with a system using voice commands instead of typing or clicking.

CODE:

```
import speech_recognition as sr
import pyttsx3
import datetime

# initialize speech engine
engine = pyttsx3.init()

def speak(text):
   print(text)
   engine.say(text)
   engine.runAndWait()

# recognizer
r = sr.Recognizer()

speak("Voice assistant started. Say add, subtract, time, or exit.")

while True:
   with sr.Microphone() as source:
       print("\nListening...")
       audio = r.listen(source)

   try:
       command = r.recognize_google(audio).lower()
       print("You said:", command)

       # addition
       if "add" in command:
           nums = [int(i) for i in command.split() if i.isdigit()]
           if len(nums) >= 2:
               result = nums[0] + nums[1]
               speak(f"The sum is {result}")

       # subtraction
       elif "subtract" in command:
           nums = [int(i) for i in command.split() if i.isdigit()]
           if len(nums) >= 2:
               result = nums[0] - nums[1]
               speak(f"The result is {result}")
```

```
    # time
    elif "time" in command:
        now = datetime.datetime.now().strftime("%H:%M")
        speak(f"The time is {now}")

    # exit
    elif "exit" in command:
        speak("Goodbye")
        break

    else:
        speak("Try saying add numbers, subtract numbers, or time")

except:
    speak("Sorry I did not understand")
```

HOW IT WORKS:

1. The program uses speech recognition to take voice input from the microphone.
2. The user's voice is converted into text using the speech recognition library.
3. The program checks the spoken command for keywords like add, subtract, time, or exit.
4. If the command contains numbers, the program performs the required calculation.
5. If the user asks for time, the program retrieves the current system time.
6. The result is converted into speech using the text-to-speech library.
7. The program speaks the result back to the user.
8. The system continues listening for commands until the user says exit.

OUTPUT:

```
(base) shrutika@Shrutikas-MacBook-Air uid % python vui.py
Voice assistant started. Say add, subtract, time, or exit.

Listening...
You said: time
The time is 11:39

Listening...
You said: add 5 and 3
The sum is 8

Listening...
You said: exit
Goodbye
(base) shrutika@Shrutikas-MacBook-Air uid %
```

RESULT:

The CLI, GUI, and VUI versions of the program were successfully implemented and tested. All interfaces performed the required operations correctly. The CLI allowed text-based interaction, the GUI provided a user-friendly graphical interface, and the VUI enabled voice-based interaction. Each interface demonstrated proper functionality and error handling.